

A Beginner-Friendly Guide for Linux / Start Learning Linux Quickly...

How to Configure and Maintain High Availability/Clustering in Linux

[Aaron Kili](#)

Last Updated: December 19, 2018

[Clustering](#)

[33 Comments](#)

High Availability (HA) simply refers to a quality of a system to operate continuously without failure for a long period of time. HA solutions can be implemented using hardware and/or software, and one of the common solutions to implementing HA is clustering.

In computing, a cluster is made up of two or more computers (commonly known as **nodes** or **members**) that work together to perform a task. In such a setup, only one node provides the service with the secondary node(s) taking over if it fails.

Clusters fall into four major types:

- **Storage:** provide a consistent file system image across servers in a cluster, allowing the servers to simultaneously read and write to a single shared file system.
- **High Availability:** eliminate single points of failure and by failing over services from one cluster node to another in case a node goes becomes inoperative.
- **Load Balancing:** dispatch network service requests to multiple cluster nodes to balance the request load among the cluster nodes.
- **High Performance:** carry out parallel or concurrent processing, thus helping to improve performance of applications.

Another widely used solution to providing **HA** is replication (specifically data

replications). Replication is the process by which one or more (secondary) databases can be kept in sync with a single primary (or master) database.

To setup a cluster, we need at least two servers. For the purpose of this guide, we will use two Linux servers:

- Node1: 192.168.10.10
- Node2: 192.168.10.11

In this article, we will demonstrate the basics of how to deploy, configure and maintain high availability/clustering in Ubuntu 16.04/18.04 and CentOS 7. We will demonstrate how to add Nginx HTTP service to the cluster.

Configuring Local DNS Settings on Each Server

In order for the two servers to communicate to each other, we need to configure the appropriate local DNS settings in the **/etc/hosts** file on both servers.

Open and edit the file using your [favorite command line editor](#).

```
$ sudo vim /etc/hosts
```

Add the following entries with actual IP addresses of your servers.

```
192.168.10.10    node1.example.com
192.168.10.11    node2.example.com
```

Save the changes and close the file.

Installing Nginx Web Server

Now install Nginx web server using the following commands.

```
$ sudo apt install nginx [On Ubuntu]
$ sudo yum install epel-release && sudo yum install nginx [On Cen
```

Once the installation is complete, start the Nginx service for now and enable it to auto-start at boot time, then check if it's up and running using the systemctl command.

On Ubuntu, the service should be started automatically immediately after package pre-configuration is complete, you can simply enable it.

```
$ sudo systemctl enable nginx
$ sudo systemctl start nginx
$ sudo systemctl status nginx
```

After starting the Nginx service, we need to create custom webpages for identifying and testing operations on both servers. We will modify the contents of the default Nginx index page as shown.

```
$ echo "This is the default page for node1.example.com" | sudo te
$ echo "This is the default page for node2.example.com" | sudo te
```

Installing and Configuring Corosync and Pacemaker

Next, we have to install **Pacemaker**, **Corosync**, and **Pcs** on each node as follows.

```
$ sudo apt install corosync pacemaker pcs          #Ubuntu
$ sudo yum install corosync pacemaker pcs          #CentOS
```

Once the installation is complete, make sure that **pcs** daemon is running on both servers.

```
$ sudo systemctl enable pcsd  
$ sudo systemctl start pcsd  
$ sudo systemctl status pcsd
```

Creating the Cluster

During the installation, a system user called “**hacluster**” is created. So we need to set up the authentication needed for **pcs**. Let’s start by creating a new password for the “**hacluster**” user, we need to use the same password on all servers:

```
$ sudo passwd hacluster
```

Create Cluster User Password

Next, on one of the servers (Node1), run the following command to set up the authentication needed for **pcs**.

```
$ sudo pcs cluster auth node1.example.com node2.example.com -u ha
```

```
[tecmint@server1 ~]$ sudo pcs cluster auth node1.example.com node2.example.com  
-u hacluster -p'=@!#@$clus' --force  
node2.example.com: Authorized  
node1.example.com: Authorized  
[tecmint@server1 ~]$
```

Setup Authentication for PCS

Now create a cluster and populate it with some nodes (the cluster name cannot exceed 15 characters, in this example, we have used **examplecluster**) on Node1 server.

```
$ sudo pcs cluster setup --name examplecluster node1.example.com
```

```
[tecmint@server1 ~]$ sudo pcs cluster setup --name examplecluster node1.example  
.com node2.example.com  
Destroying cluster on nodes: node1.example.com, node2.example.com...  
node2.example.com: Stopping Cluster (pacemaker)...  
node1.example.com: Stopping Cluster (pacemaker)...  
node2.example.com: Successfully destroyed cluster  
node1.example.com: Successfully destroyed cluster  
  
Sending 'pacemaker_remote authkey' to 'node1.example.com', 'node2.example.com'  
node2.example.com: successful distribution of the file 'pacemaker_remote authkey'  
node1.example.com: successful distribution of the file 'pacemaker_remote authkey'  
Sending cluster config files to the nodes...
```

```
node1.example.com: Succeeded
node2.example.com: Succeeded

Synchronizing pcsd certificates on nodes node1.example.com, node2.example.com...
node2.example.com: Success
node1.example.com: Success
Restarting pcsd on the nodes in order to reload the certificates...
node2.example.com: Success
node1.example.com: Success
[tecmint@server1 ~]$
```

Create Cluster on Node1

Now enable the cluster on boot and start the service.

```
$ sudo pcs cluster enable --all
$ sudo pcs cluster start --all
```

Enable and Start the Cluster

Now check if the cluster service is up and running using the following command.

```
$ sudo pcs status  
OR  
$ sudo crm_mon -1
```

```
[tecmint@server1 ~]$ sudo pcs status  
Cluster name: examplecluster  
  
WARNINGS:  
No stonith devices and stonith-enabled is not false  
  
Stack: corosync  
Current DC: node2.example.com (version 1.1.19-8.el7_6.1-c3c624ea3d) - partition WITHOUT quorum  
Last updated: Wed Dec 19 01:21:38 2018  
Last change: Wed Dec 19 01:18:51 2018 by hacluster via crmd on node2.example.com  
  
2 nodes configured  
0 resources configured  
  
Node node1.example.com: UNCLEAN (offline)  
Online: [ node2.example.com ]  
  
No resources  
  
Daemon Status:  
  corosync: active/enabled  
  pacemaker: active/enabled  
  pcsd: active/enabled  
[tecmint@server1 ~]$
```

Check Cluster Status

From the output of the above command, you can see that there is a warning about no **STONITH** devices yet the **STONITH** is still enabled in the cluster. In addition, no cluster resources/services have been configured.

Configuring Cluster Options

The first option is to disable **STONITH** (or **Shoot The Other Node In The Head**), the fencing implementation on **Pacemaker**.

This component helps to protect your data from being corrupted by concurrent access. For the purpose of this guide, we will disable it since we have not configured any devices.

To turn off **STONITH**, run the following command:

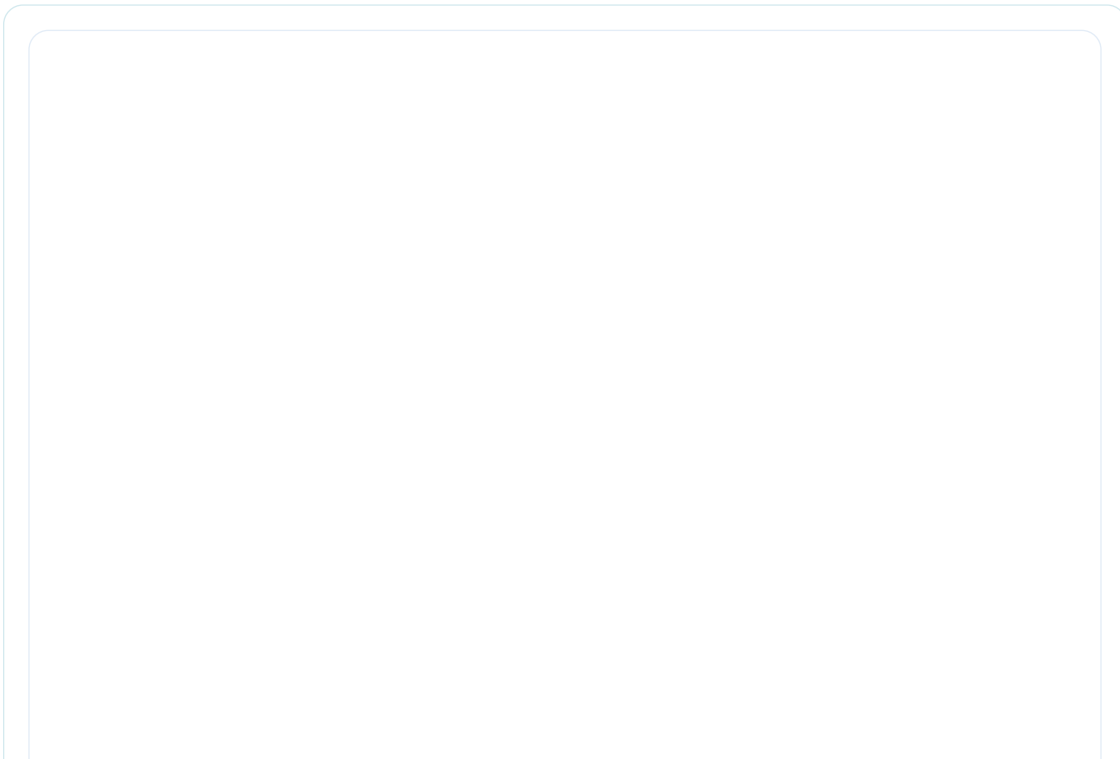
```
$ sudo pcs property set stonith-enabled=false
```

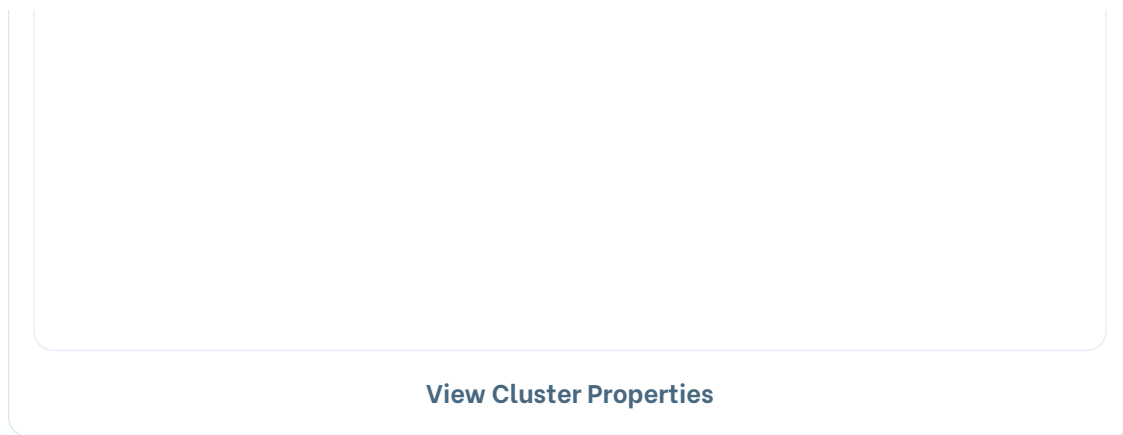
Next, also ignore the **Quorum** policy by running the following command:

```
$ sudo pcs property set no-quorum-policy=ignore
```

After setting the above options, run the following command to see the property list and ensure that the above options, **stonith** and the **quorum policy** are disabled.

```
$ sudo pcs property list
```





Adding a Resource/Cluster Service

In this section, we will look at how to add a cluster resource. We will configure a floating IP which is the IP address that can be instantly moved from one server to another within the same network or data center. In short, a floating IP is a technical common term, used for IPs which are not bound strictly to one single interface.

In this case, it will be used to support failover in a high-availability cluster. Keep in mind that floating IPs aren't just for failover situations, they have a few other use cases. We need to configure the cluster in such a way that only the active member of the cluster "owns" or responds to the floating IP at any given time.

We will add two cluster resources: the floating IP address resource called "**floating_ip**" and a resource for the Nginx web server called "**http_server**".

First start by adding the floating_ip as follows. In this example, our floating IP address is **192.168.10.20**.

```
$ sudo pcs resource create floating_ip ocf:heartbeat:IPaddr2 ip=1
```

where:

- **floating_ip**: is the name of the service.
- **"ocf:heartbeat:IPaddr2"**: tells Pacemaker which script to use, IPaddr2 in this case, which namespace it is in (pacemaker) and what standard it conforms to ocf.

- “**op monitor interval=60s**”: instructs Pacemaker to check the health of this service every one minutes by calling the agent’s monitor action.

Then add the second resource, named **http_server**. Here, resource agent of the service is **ocf:heartbeat:nginx**.

```
$ sudo pcs resource create http_server ocf:heartbeat:nginx config
```

Once you have added the cluster services, issue the following command to check the status of resources.

```
$ sudo pcs status resources
```

Check Cluster Resources

Looking at the output of the command, the two added resources: “**floating_ip**” and “**http_server**” have been listed. The `floating_ip` service is off because the primary node is in operation.

If you have firewall enabled on your system, you need to allow all traffic to **Nginx** and all high availability services through the firewall for proper communication between nodes:

```
----- CentOS 7 -----  
$ sudo firewall-cmd --permanent --add-service=http  
$ sudo firewall-cmd --permanent --add-service=high-availability  
$ sudo firewall-cmd --reload  
  
----- Ubuntu -----  
$ sudo ufw allow http  
$ sudo ufw allow high-availability  
$ sudo ufw reload
```

Testing High Availability/Clustering

The final and important step is to test that our high availability setup works. Open a web browser and navigate to the address **192.168.10.20** you should see the default Nginx page from the **node2.example.com** as shown in the screenshot.

Test Cluster Before Failure

To simulate a failure, run the following command to stop the cluster on the **node2.example.com**.

```
$ sudo pcs cluster stop http_server
```

Then reload the page at **192.168.10.20**, you should now access the default Nginx web page from the **node1.example.com**.

Test Cluster After Failure

Alternatively, you can simulate an error by telling the service to stop directly, without stopping the the cluster on any node, using the following command on one of the nodes:

```
$ sudo crm_resource --resource http_server --force-stop
```

Then you need to run **crm_mon** in interactive mode (the default), within the monitor interval of 2 minutes, you should be able to see the cluster notice that **http_server** failed and move it to another node.

For your cluster services to run efficiently, you may need to set some constraints. You can see the **pcs** man page (`man pcs`) for a list of all usage commands.

For more information on Corosync and Pacemaker, check out: <https://clusterlabs.org/>

Summary

In this guide, we have shown the basics of how to deploy, configure and maintain high availability/clustering/replication in Ubuntu 16.04/18.04 and CentOS 7. We demonstrated how to add Nginx HTTP service to a cluster. If you have any thoughts to share or questions, use the feedback form below.

🔖 [High Availability](#)

If you read this far, tweet to the author to show them you care.

[Tweet a thanks](#)

PREVIOUS ARTICLE:

How to Clone a Partition or Hard drive in Linux

NEXT ARTICLE:

How to Install pgAdmin4 in CentOS 7



Aaron Kili

Aaron Kili is a Linux and F.O.S.S enthusiast, an upcoming Linux SysAdmin, web developer, and currently a content creator for TecMint who loves working with computers and strongly believes in sharing knowledge.

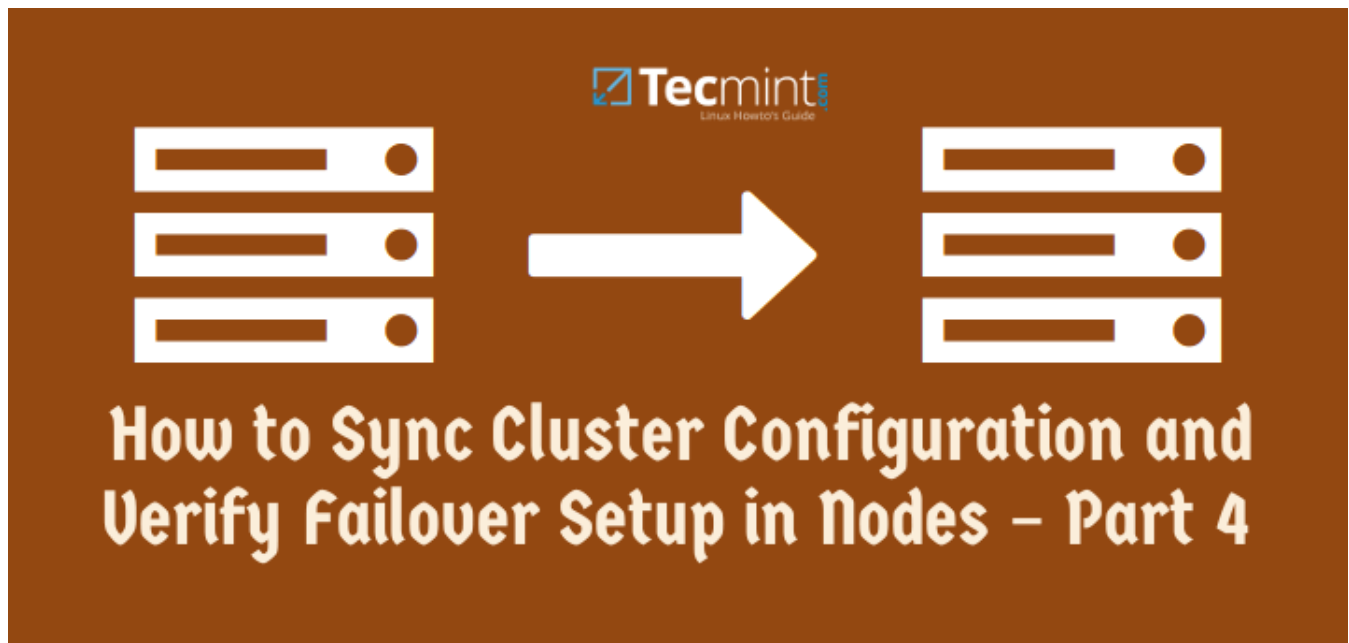
*Each tutorial at **TecMint** is created by a team of experienced Linux system administrators so that it meets our high-quality standards.*

Join the [TecMint Weekly Newsletter](#) (More Than **156,129** Linux Enthusiasts Have Subscribed)

Was this article helpful? Please [add a comment](#) or [buy me a coffee](#) to show your appreciation.

Related Posts

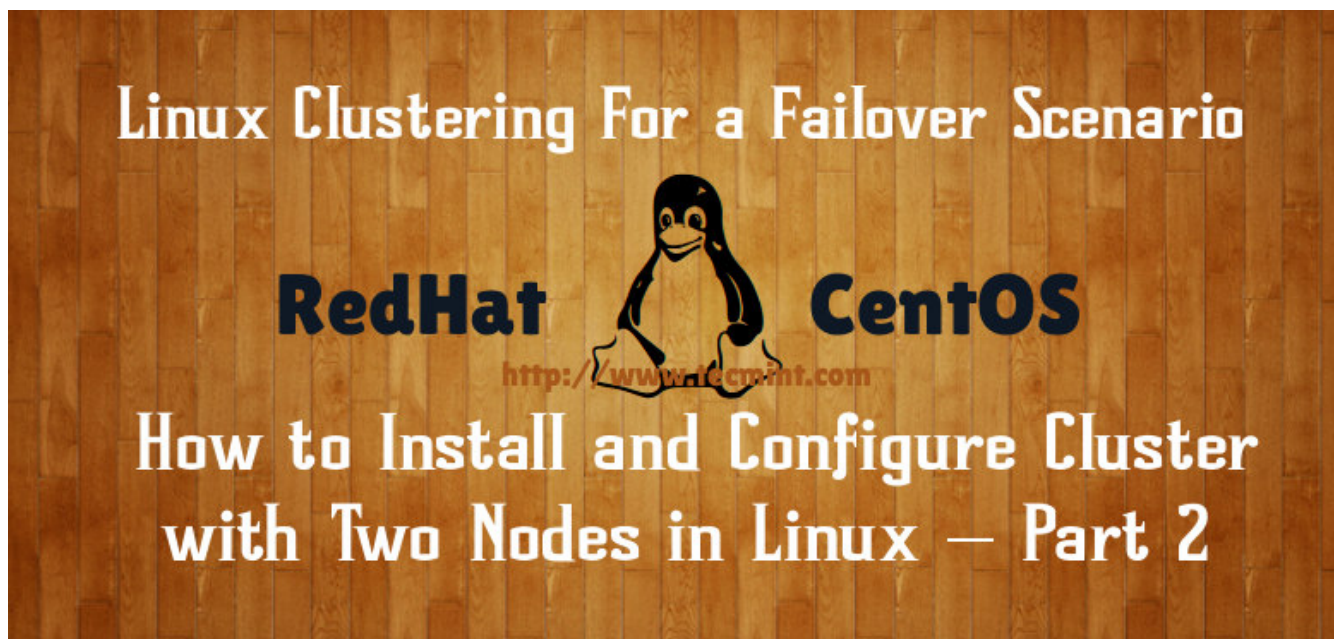
What is QUORUM Disk and a Fencing Wars?



[How to Sync Cluster Configuration and Verify Failover Setup in Nodes – Part 4](#)



[Fencing and Adding a Failover to Clustering – Part 3](#)



[How to Install and Configure Cluster with Two Nodes in Linux – Part 2](#)



[Introduction and Advantages/Disadvantages of Clustering in Linux – Part 1](#)

[33 thoughts on “How to Configure and Maintain High Availability/Clustering in Linux”](#)

[← Older Comments](#)**zain**

December 20, 2022 at 1:27 am

Hi Aaron,

I am getting this output..

```
$ sudo pcs status resources
$ sudo pcs cluster stop http_server
```

```
floating_ip (ocf::heartbeat:IPaddr2): Started hacluster1
http_server (ocf::heartbeat:nginx): Started hacluster2
Error: nodes 'http_server' do not appear to exist in
configuration
```

In that case, how do I test, I am not able to switch between these.

Please help.

Regards,
zain

[Reply](#)**Kim**

July 17, 2021 at 10:13 am

```
root@Application-node-01:~# sudo pcs cluster setup --name ha_lecluster  
mofapp1.local.la
```

Error: Specified option '--name' is not supported in this command

Hint: Syntax has changed from the previous version. See 'man pcs' → Changes in pcs-0.10.

Please help

[Reply](#)

Kim

[July 17, 2021 at 10:15 am](#)

I used Ubuntu 20.04.

[Reply](#)

Simone24c

[December 28, 2021 at 5:05 pm](#)

Try `sudo pcs cluster setup ha_lecluster mofapp1.local.la`

[Reply](#)

sravan kumar s

[November 12, 2020 at 4:22 pm](#)

Hi, Thanks for the article,

Is this HA is suitable for ntp service? if yes can you provide the detailed commands to add the ntp service?

Thanks, And Regards,

Sravan

[Reply](#)

Suyash

[September 7, 2020 at 7:37 pm](#)

Hi,

Kindly please help me understand, Why we have taken floating IP as 192.169.10.20? as our node1 and node2 have (Node1: 192.168.10.10 / Node2: 192.168.10.1).

Thanks in advance.

Suyash

[Reply](#)

Leandro Aurelio

[July 13, 2021 at 3:30 am](#)

I'm not 100% about my answer, but we have 10.10 and 10.1 as nginx client,

and for the front end the address will be 10.20, it means that our servers can have any ip address, the browser will always receive the requests as 192.168.10.20 and nginx will forward the requests to Node1: 192.168.10.10 or Node2: 192.168.10.1 depending on the availability.

[Reply](#)

Ulundo

[August 20, 2020 at 9:13 pm](#)

Hello, I did the test and gave me the following command error in Ubuntu 20.

The command is this:

```
$ sudo pcs cluster auth node1.example.com node2.example.com  
-u hacluster -p 'tcc2020' -force
```

Error: Specified options '-force' is not supported on this command .. What do I do?

[Reply](#)



Aaron Kili

[August 24, 2020 at 10:05 am](#)

@Ulundo

Try to use `--force` instead of `-force`.

[Reply](#)

marius

August 31, 2020 at 9:47 pm

Doesn't work for work...

[Reply](#)

Suresh

July 12, 2020 at 6:14 pm

Hi,

Please clarify the below,

1. Does it require nginx Plus?
2. How to map that virtual IP to resolve DNS name, bit confused how to use IP in production.

Thanks in Advance!

[Reply](#)



Aaron Kili

July 13, 2020 at 11:14 am

@Suresh

This does not require NGINX Plus and maybe you need to read more about IP addressing and subnetting and how to do it in a production environment.

[Reply](#)

Taoufik

October 19, 2019 at 7:43 pm

Hello,

As I know the STONITH should not be disabled in production. I don't think this article is recommended to use in production.

[Reply](#)

pipo

October 10, 2019 at 10:26 am

Hello. I wrote down because I had a question.

I want to check the failover time of the pacemaker. The failover time that I mean is the time when **node1** is down and **node2** is up.

I wonder if there are any data provided by the pacemaker about the failover time. If not, how can I check the failover time?

I wrote the source code with Python, and I wrote the `sudo pcs cluster stop node1` into the console window, and then I took the time.

In **node2**, the **node1** was monitored through the `sudo pcs status` and `sudo crm_mon -1`, and measured the time if the **node1** went offline.

But when I measured the time, I was wondering if I could measure it this way because it's longer than I thought.

I'm curious about your answer. Please reply.

[Reply](#)

pippin Han

August 12, 2019 at 3:58 pm

[solved] first, In my case the same symptom occurred: floating_ip consistently runs on serverA while http_server runs on serverB.

It was a very difficult situation and the secret was in grouping...

```
$ sudo crm configure group hakase_balancing floating_ip  
http_server
```

That's It! running fine without any issues..

[Reply](#)

**Aaron Kili**

August 19, 2019 at 2:34 pm

@Pippin

Thanks a lot for sharing this solution. It's a great addition to this guide.

[Reply](#)**Mark**

August 5, 2019 at 11:45 pm

Search...

needs to be opened to properly jump start the cluster. Also, I ran through this twice and the command “`sudo pcs cluster stop http_server`” tells me “nodes ‘**http_server**’ do not appear to exist in configuration.

In addition to the above issues, **floating_ip** consistently runs on **serverA** while **http_server** runs on **serverB**. So the site is offline at the floating IP and at the IP of the server that has the floating ip.

[Reply](#)[← Older Comments](#)

Got something to say? Join the discussion.

Have a question or suggestion? Please leave a comment to start the discussion.

[A Beginners Guide To Learn Linux for Free \[with Examples\]](#)

[Red Hat RHCSA/RHCE 8 Certification Study Guide \[eBooks\]](#)

[Linux Foundation LFCS and LFCE Certification Study Guide \[eBooks\]](#)

Linux Commands and Tools

[Linux sdiff Command Examples for Linux Newbies](#)

[Pssh – Execute Commands on Multiple Remote Linux Servers Using Single Terminal](#)

[How to Auto Execute Commands/Scripts During Reboot or Startup](#)

[21 Tar Command Examples in Linux](#)

[45 Zypper Commands to Manage ‘Suse’ Linux Package Management](#)

[10 Interesting Linux Command Line Tricks and Tips Worth Knowing](#)

Linux Server Monitoring Tools

[How to Install Nagios Core in Rocky Linux and AlmaLinux](#)

[Linfo – Shows Linux Server Health Status in Real-Time](#)

[Bashtop – A Resource Monitoring Tool for Linux](#)

[Watchman – A File and Directory Watching Tool for Changes](#)

[How to Install htop on CentOS 8](#)

[iftop – A Real Time Linux Network Bandwidth Monitoring Tool](#)

Learn Linux Tricks & Tips

[How to Check Which Apache Modules are Enabled/Loaded in Linux](#)

[How To Assign Output of a Linux Command to a Variable](#)

[How to Add a New Disk to an Existing Linux Server](#)

[vlock – A Smart Way to Lock User Virtual Console or Terminal in Linux](#)

[4 Useful Way to Know Plugged USB Device Name in Linux](#)

[How to Test Website Loading Speed in Linux Terminal](#)

Best Linux Tools

[7 Useful Linux Security Features and Tools for Beginners](#)

[25 Free Open Source Applications I Found in Year 2022](#)

[My Favorite Command Line Editors for Linux – What’s Your Editor?](#)

[9 Best Google Drive Clients for Linux in 2023](#)

[9 Best Free UPnP and DLNA Media Servers for Linux](#)

[The 5 Best Command Line Music Players for Linux](#)

Tecmint: Linux Howtos, Tutorials & Guides © 2023. All Rights Reserved.

The material in this site cannot be republished either online or offline, without our permission.

Hosting Sponsored by : [Linode Cloud Hosting](#)